

Fiesta de panqueques

Bohan es el anfitrión de una fiesta. Como le encantan los panqueques, decide comprarlos de la tienda cercana para su fiesta. La tienda vende N sabores distintos de panqueques, numerados del 0 al $N - 1$. Bohan decide comprar N panqueques de cada sabor, o sea, N^2 panqueques en total. Cada panqueque se divide en K rebanadas idénticas antes de ser colocado en su propia bolsa. Las bolsas están numeradas del 0 al $N^2 - 1$. Sea $F[i]$ el sabor del panqueque de la bolsa i .

Durante la fiesta, Bohan invita a las personas presentes a jugar un juego. El juego procede de la siguiente forma.

Para empezar, Bohan escoge N invitados y los lleva a cuartos numerados $0, 1, \dots, N - 1$ de tal manera que en cada cuarto se encuentra exactamente un invitado. Los invitados restantes permanecen afuera hasta el final del juego. Como la apariencia de los N cuartos es idéntica, los invitados no saben en qué cuarto están.

Después, para cada cuarto i ($0 \leq i < N$), Bohan entra al cuarto i y le dice en secreto al invitado un entero $P[i]$ que representa un **sabor prohibido**. Puede o no también decirle al invitado en qué cuarto está. Bohan garantiza que $[P[0], P[1], \dots, P[N - 1]]$ es una permutación de $[0, 1, \dots, N - 1]$.

Luego, se llevan a cabo N rondas. En la i -ésima ronda, Bohan trae las bolsas $0, 1, \dots, N^2 - 1$ una por una en orden secuencial al cuarto $i - 1$. Cuando el invitado en el cuarto $i - 1$ recibe la bolsa j , obtiene la siguiente información:

- $F[j]$, el sabor del panqueque en la bolsa j , y
- la cantidad de rebanadas que quedan en la bolsa.

Antes de recibir la siguiente bolsa, el invitado en el cuarto $i - 1$ debe decidir cuántas rebanadas se va a comer de la bolsa actual. La cantidad de rebanadas que se puede comer depende de la situación:

- Si $P[i - 1] \neq F[j]$, el invitado puede sacar cualquier cantidad de rebanadas de la bolsa y comérselas.
- Si $P[i - 1] = F[j]$, el invitado no puede comerse ninguna rebanada de la bolsa.

Cabe notar que los invitados no conocen la secuencia $F[0], F[1], \dots, F[N^2 - 1]$ por adelantado.

Después de las N rondas, los invitados que están afuera reciben la secuencia de bolsas. Al ver las bolsas, averiguan los sabores de los panqueques y la cantidad de rebanadas que quedan en cada

bolsa. Con esta información deben determinar los valores de $P[0], P[1], \dots, P[N - 1]$.

Los invitados tienen permitido comunicarse antes de que empiece el juego. En ese momento también conocen los valores de N y K . Tu objetivo es implementar una estrategia para que los invitados que se quedan afuera siempre puedan determinar los valores de $P[0], P[1], \dots, P[N - 1]$ correctamente.

Detalles de implementación

Necesitas implementar tres rutinas.

Deberás implementar las siguientes dos rutinas para los invitados en los cuartos $0, 1, \dots, N - 1$.

```
void init(int N, int K, int p, int r)
```

Supón que el invitado está en el cuarto i .

- N : la cantidad de sabores de panqueques.
- K : la cantidad de rebanadas de cada panqueque antes de empezar el juego.
- $p = P[i]$: el sabor prohibido en el cuarto i .
- Si Bohan decide indicarle al invitado en qué cuarto está, entonces $r = i$. De otro modo, $r = -1$.

```
int strategy(int b, int f, int s)
```

Supón que el invitado está en el cuarto i .

- b : el número de la bolsa actual.
- $f = F[b]$: el sabor del panqueque en la bolsa b .
- s : la cantidad de rebanadas que quedan en la bolsa b .
- Esta rutina debe devolver un entero no negativo x que representa la cantidad de rebanadas que el invitado decide comerse.
 - Si $f \neq P[i]$, entonces se debe cumplir que $0 \leq x \leq s$.
 - Si $f = P[i]$, entonces se debe cumplir que $x = 0$.

Deberás implementar la siguiente rutina para los invitados que están afuera.

```
std::vector<int> guess(int N, int K, std::vector<int> F,  
                      std::vector<int> S)
```

- N : la cantidad de sabores de panqueques.
- K : la cantidad de rebanadas de cada panqueque antes de empezar el juego.

- F : un arreglo de longitud N^2 en el que $F[i]$ es el sabor del panqueque en la bolsa i .
- S : un arreglo de longitud N^2 en el que $S[i]$ es la cantidad de rebanadas que quedan en la bolsa i después de las N rondas.
- Este procedimiento deberá devolver un arreglo P de longitud N en el que $P[i]$ es el número que se le dio al invitado en el cuarto i .

Cada caso de prueba involucra T juegos independientes.

Durante el proceso de evaluación, para cada uno de los T juegos, habrá $N + 1$ procesos. Para cada i tal que $0 \leq i < N$, el proceso i representa al invitado en el cuarto i . El proceso N representa a los invitados que están afuera. Para cada i tal que $0 \leq i < N$, el proceso $(i + 1)$ comienza después de que el proceso i termina.

Para los procesos del 0 al $N - 1$:

- `init` se llama exactamente una vez.
- `strategy` se llama N^2 veces después de `init`.
 - Se garantiza que en la j -ésima llamada, $b = j - 1$.

Para el proceso N :

- `guess` se llama exactamente una vez.

El juez puede intercalar procesos de juegos distintos, pero se garantiza que se preserva el orden relativo de los procesos dentro de cada juego individual.

Consideraciones

- $1 \leq T \leq 400$.
- $2 \leq N \leq 30$.
- La suma de N^3 sobre todos los juegos en un caso de prueba no pasa de 27 000.
- $1 \leq K \leq N$
- $K \in \{1, 3, N\}$.
- $[P[0], P[1], \dots, P[N - 1]]$ es una permutación de $[0, 1, \dots, N - 1]$.
- $0 \leq F[j] < N$ para cada j tal que $0 \leq j < N^2$.
- i aparece exactamente N veces en $[F[0], F[1], \dots, F[N^2 - 1]]$ para cada i tal que $0 \leq i < N$.
- El juez **no es adaptativo**, o sea, $[P[0], P[1], \dots, P[N - 1]]$ y $[F[0], F[1], \dots, F[N^2 - 1]]$ se fijan antes de la primera llamada a `init`.

Subtareas

Subtarea	Puntuación	Restricciones adicionales
1	8	$K = 1, N = 2$.
2	7	$K = 1$ y Bohan decide que les va a decir a todos en qué cuarto están.
3	14	$K = 1$ y $[F[i \cdot N], F[i \cdot N + 1], \dots, F[i \cdot N + N - 1]]$ es una permutación de $[0, 1, \dots, N - 1]$ para cada i tal que $0 \leq i < N$.
4	18	$K = N$.
5	21	$K = 3, N \geq 3$.
6	32	$K = 1$.

Ejemplo

Considera la situación en la que $T = 1$ y $N = 2, K = 2, P = [1, 0], F = [1, 0, 0, 1]$ para el único juego en el caso de prueba.

Sea $S[i]$ la cantidad actual de rebanadas que quedan en la bolsa i . Inicialmente, $S = [2, 2, 2, 2]$.

Supón que los invitados fijan la siguiente estrategia antes de que el juego comience.

- Para los invitados en los cuartos, si tienen permitido comerse el panqueque actual:
 - Si el panqueque actual es el primero que se pueden comer, entonces se comerán todas las rebanadas que quedan.
 - De no ser así, solo se comerán una rebanada.
- Para los invitados que están afuera:
 - Si $S = [0, 0, 1, 1]$, entonces adivinarán que $P = [1, 0]$.
 - De no ser así, adivinarán que $P = [0, 1]$.

El juego procede de la siguiente forma.

Primero, el juez inicia el proceso 0 que representa al invitado en el cuarto 0 y llama `init(2, 2, 1, -1)`. Después de la inicialización, el juez hace las siguientes llamadas:

Llamada	Valor retornado
<code>strategy(0, 1, 2)</code>	0
<code>strategy(1, 0, 2)</code>	2
<code>strategy(2, 0, 2)</code>	1
<code>strategy(3, 1, 2)</code>	0

Tanto la primera llamada como la cuarta deben retornar 0 porque $P[0] = F[0] = F[3] = 1$.

Después de que este proceso termina (la ronda 1 acaba), $S = [2, 0, 1, 2]$.

Luego, el juez inicia el proceso 1 que representa al invitado 1 y llama `init(2, 2, 0, -1)`. Después de la inicialización, el juez hace las siguientes llamadas:

Llamada	Valor retornado
<code>strategy(0, 1, 2)</code>	2
<code>strategy(1, 0, 0)</code>	0
<code>strategy(2, 0, 1)</code>	0
<code>strategy(3, 1, 2)</code>	1

Tanto la segunda llamada como la tercera deben retornar 0 pues $P[1] = F[1] = F[2] = 0$.

Después de que este proceso termina (la ronda 2 acaba), $S = [0, 0, 1, 1]$.

Finalmente, el juez inicia el proceso 2 que representa a los invitados de afuera. El juez hace la siguiente llamada:

Llamada	Valor retornado
<code>guess(2, 2, [1, 0, 0, 1], [0, 0, 1, 1])</code>	<code>[1, 0]</code>

Los invitados de afuera adivinaron la permutación correcta. Por lo tanto, este caso se prueba se considera correcto.

Es importante notar que esta estrategia no siempre le permite a los invitados de afuera adivinar la permutación correctamente.

Además de este ejemplo, el archivo adjunto de esta tarea contiene también una entrada de ejemplo distinta.

Evaluador de ejemplo

El evaluador de ejemplo solo soporta **un juego por caso de prueba**.

Formato de la entrada:

```
N K
P[0] P[1] ... P[N-1]
F[0] F[1] ... F[N*N-1]
revelar
```

- `revelar` es 0 o 1.
 - Si `revelar` es 0, el evaluador actuará como si Bohan no le dice a nadie su número de cuarto.
 - Si `revelar` es 1, el evaluador actuará como si Bohan les dice a todos sus números de cuarto.

Si el arreglo que regresa `guess` concuerda con $[P[0], P[1], \dots, P[N - 1]]$, el evaluador de ejemplo imprime

```
Accepted
```

Así mismo, el evaluador imprime las llamadas que se hicieron en orden con el propósito de debuggeo.